GitHub Copilot 워크플로 과정

사용량 · Harness

개념 → 바로 실습 · 사용량 읽기 · 낭비 패턴 · Hooks · 대화 리셋

1 사용량 읽기
▸ 실습 1

2 낭비 패턴
▸ 실습 2

3 Hooks
▸ 실습 3

4 대화 리셋
▸ 실습 4

5 마무리
▸ 실습 5

01

내 사용량 읽기

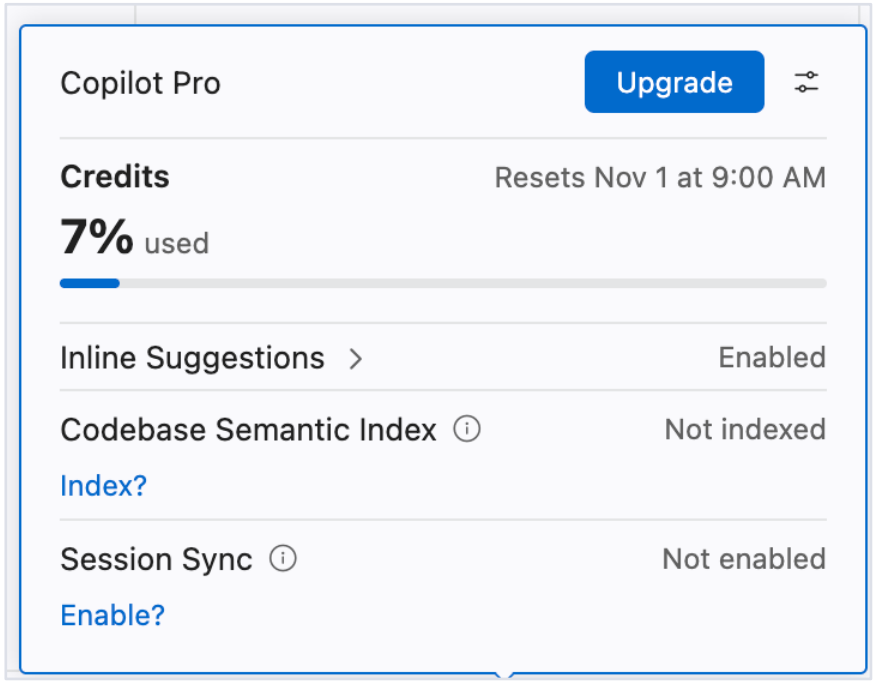
요청별 · 세션별 · 월별 사용량을 어디서 보는지 확인하고 바로 기록합니다.

사용량을 보는 곳

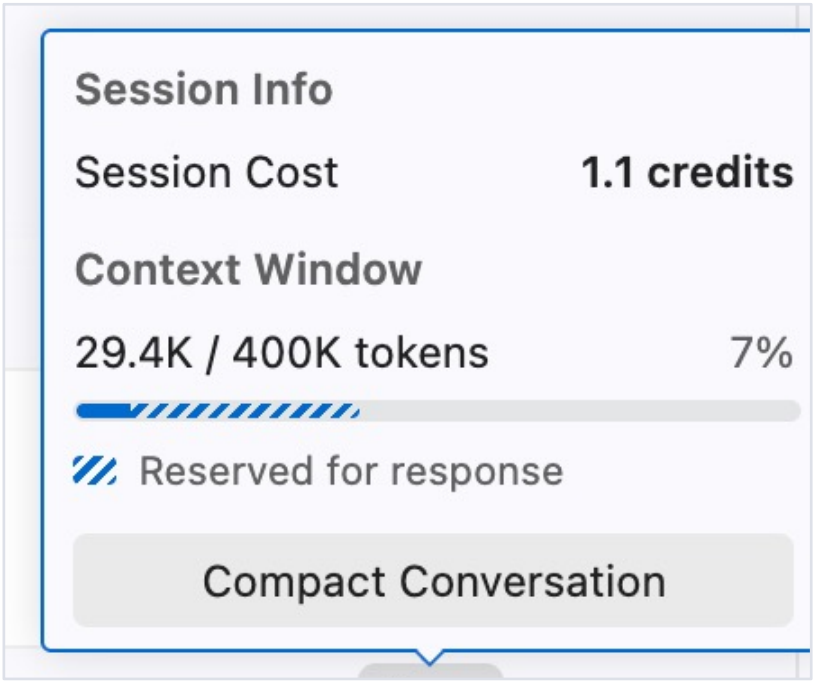
범위	어디서	무엇이 보이나
요청 1건	응답 맨 아래 `시각 · 모델 · N credits`	처리 모델과 비용
세션 1개	입력창 아래 원형 아이콘(6%) → **Session Info**	누적 비용 · 토큰 · Compact 버튼
이번 달	상태 표시줄 **Copilot 아이콘**	`Credits N% used`, 초기화 날짜
이번 달 (웹)	프로필 → **Copilot settings** → **Usage**	Business/Enterprise 이번 주기 사용량
개인 플랜	`github.com/settings/billing` → **AI usage**	초과 사용량 · 모델별 크레딧
CLI	`/usage`, `/context`	세션 사용량, 현재 컨텍스트

캐시에서 다시 쓰는 토큰은 보통 입력 가격의 10%

화면에서 보는 위치 — 이번 달 · 이번 세션



상태 표시줄 Copilot 아이콘 → 이번 달 사용률 · 초기화 날짜



입력창 아래 원형 아이콘 → Session Info

1 **Credits 7% used**
이번 달 포함 크레딧 중 쓴 비율

2 **Session Cost**
이 대화에서 지금까지 쓴 크레딧

3 **Context Window**
지금 대화가 차지한 토큰 수

실습

실습 1 · 내 사용량 확인

이번 달 사용률과 대화 하나의 비용·컨텍스트를 직접 읽고
공통 정리 시간의 기준선과 비교해 기록합니다.

준비

- VS Code + Copilot 로그인
- 공통 정리 시간에 적은 기준선 메모

실습 1

내 사용량 확인

1. VS Code 오른쪽 아래 **Copilot 아이콘** → `Credits N% used` 확인

2. 지금까지 쓴 대화 하나 열기

3. 입력창 아래 **원형 아이콘** → Session Info 의 비용 · 컨텍스트 확인

4. 공통 정리 시간에 적은 **기준선**과 비교해 기록

✅ **이번 달 % 기록** ✅ **대화 1개의 credits · 토큰 기록** ✅ **기준선과 차이 한 줄**

02

토큰 낭비 패턴

토큰이 새는 7가지 길과 처방 — 그중 ③을 실습으로 확인합니다.

어디서 새고, 어떻게 막나

#	낭비	처방
①	대화를 끝없이 이어감	작업이 바뀌면 **새 대화**, 넘길 것은 3줄 요약만
②	`#codebase` · 전체 검색 습관	아는 파일은 `+` 로 **첨부**
③	에러 붙여 넣고 "고쳐"만 반복	가설·범위·완료 조건, **2회 실패 시 멈춤**
④	장문 일반론 instructions	고유 규칙만 40줄 이내, 긴 절차는 Skill
⑤	파일 20개 탐색	Instructions에 **진입점**, 요청에 **범위**
⑥	Autopilot에 통째로 맡김	횟수·크레딧 제한 + **Harness** + diff 확인
⑦	모델 과잉 / 대화 중 모델 변경	**Auto 기본**, 바꿀 땐 새 대화에서

실습

실습 2 · 낭비 ③ 처방 비교

같은 실패를 "고쳐"와 가설·범위·완료 조건 프롬프트로 각각 고치게 하고
단계 수 · 비용 · 바꾼 파일을 비교합니다.

준비

- 지금 브랜치 그대로 (M3 를 못 끝냈으면 `m4-start`)
- 모델 GPT-6 Luna 고정 · Agent · Manual permissions
- (a)·(b) 각각 새 대화, 끝나면 `git checkout -- src tests`

실습 2-0 ①

시작 전 ① 브랜치 확인

1. `git status` — 커밋 안 한 변경이 없는지 확인
2. M3 를 마쳤으면 지금 브랜치 그대로 (체크아웃 불필요)
3. M3 를 못 끝냈으면 오른쪽 아래 3줄 실행

명령 (터미널, 순서대로)

```
# 상태 확인
git status

# M3 를 못 끝낸 경우에만
git stash -u
git checkout m4-start
python -m pip install -r requirements.txt
```

✅ **`nothing to commit, working tree clean`**

실습 2-0 ②

시작 전 ② 실패를 내 눈으로 확인

1. UTC 시간대로 테스트 → **1 failed** 확인
2. 이 실패 출력을 복사해 두기 — (a) 프롬프트에 붙임
3. 로컬 시간대로 테스트 → 모두 passed

명령 (터미널)

```
# UTC 시간대 — Windows PowerShell
$env:TZ="UTC0"; python -m pytest -q; Remove-Item Env:TZ

# UTC 시간대 — mac/Linux
TZ=UTC python -m pytest -q

# 로컬 시간대
python -m pytest -q
```

✅ **UTC: `test_day_range_includes_early_morning_kst` 1 failed** ✅ **로컬: 모두 passed**

실습 2-1

(a) 에러만 붙여 넣고 "고쳐"

1. 새 대화(+) 열기 · 모델 GPT-6 Luna 확인
2. 오른쪽 프롬프트 + 내 실패 출력 붙여 보내기
3. 끝나면 UTC · 로컬 테스트 다시 실행 (시작 전 ② 명령)
4. 단계 수 · Session Info 비용 · 바꾼 파일(`git status`) 기록
5. `git checkout -- src tests` 로 되돌리기

프롬프트 (a)

시간대를 UTC 로 두고 테스트를 돌리니
아래처럼 실패한다.
고쳐.

```
FAILED tests/test_report.py::
test_day_range_includes_early_morning_kst
E AssertionError: assert
  datetime.datetime(2026, 9, 21, 0, 0,
  tzinfo=…UTC)
  <= datetime.datetime(2026, 9, 21, 5, 0,
  tzinfo=…Asia/Seoul)
1 failed, 32 passed
```

실습 2-2

(b) 가설 · 범위 · 완료 조건 · 멈춤 규칙

1. 다시 새 대화(+) 열기 → 오른쪽 프롬프트 보내기
2. 끝나면 UTC · 로컬 테스트 다시 실행 (시작 전 ② 명령)
3. (a)와 같은 항목 기록
4. ✅ 단계 절반 · 비용 40%↓ · 소스만 수정
5. `git checkout -- src tests` 로 되돌리기

프롬프트 (b)

```
UTC 시간대에서 tests/test_report.py::
test_day_range_includes_early_morning_kst
가 실패한다.
가설: report/uptime.py 의
day_range 가 .astimezone() 으로
실행 머신의 로컬 시간대를 쓴다.
일별 리포트는 Asia/Seoul 기준이어야 한다.
범위: day_range 만 고치고
테스트 파일은 바꾸지 마.
완료 조건: python -m pytest -q 가 UTC
시간대(Windows PowerShell:
$env:TZ="UTC0", mac: TZ=UTC)와 로컬
시간대 둘 다 통과.
2회 시도해도 안 되면 멈추고 보고해라.
```

실행 결과 — (a) 테스트까지 고침 vs (b) 소스만



Completed 20 steps in 58s +10 -5

수정했습니다. `day_range()` 가 실행 환경의 로컬 시간대에 의존하지 않고, 날짜를 KST 기준으로 계산한 뒤 UTC 경계로 반환합니다. 2026-09-21 범위가 `[2026-09-20 15:00 UTC, 2026-09-21 15:00 UTC)` 인지 확인하는 회귀 검증도 추가했습니다.

변경 파일:

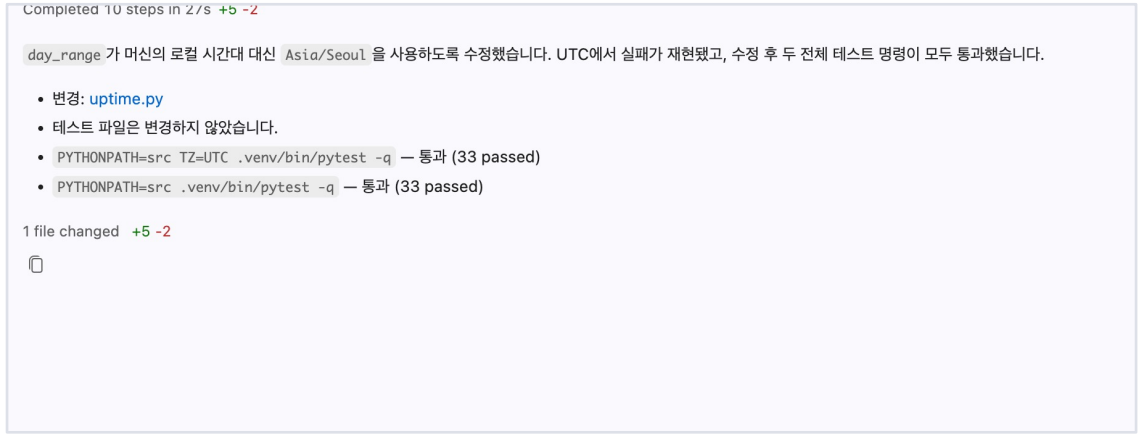
- uptime.py
- test_report.py

검증 결과:

- `PYTHONPATH=src TZ=UTC .venv/bin/python -m pytest -q` — **33 passed**
- `PYTHONPATH=src TZ=UTC .venv/bin/python -m pytest -q tests/test_report.py` — **4 passed**

`PYTHONPATH` 없이 전체 테스트를 실행하면 가상환경이 옆 저장소의 소스를 가져와 테스트 수집 단계에서 실패했습니다. 현재 worktree의 소스를 명시해 재실행한 결과 전체 테스트가 통과했습니다.

(a) 20 steps · 0.7 credits · `test_report.py` 도 수정



Completed 10 steps in 27s +5 -2

`day_range` 가 머신의 로컬 시간대 대신 `Asia/Seoul` 을 사용하도록 수정했습니다. UTC에서 실패가 재현됐고, 수정 후 두 전체 테스트 명령이 모두 통과했습니다.

- 변경: uptime.py
- 테스트 파일은 변경하지 않았습니다.
- `PYTHONPATH=src TZ=UTC .venv/bin/pytest -q` — 통과 (33 passed)
- `PYTHONPATH=src .venv/bin/pytest -q` — 통과 (33 passed)

1 file changed +5 -2

(b) 10 steps · 0.4 credits · `uptime.py` 만

1 탐색을 사람이 대신

단계 절반, 비용 약 40% 감소

2 범위가 더 큰 차이

"테스트를 바꿔 통과" 습관 차단

3 2회 실패 시 멈춤

반복 실패 루프를 끊는 안전장치

⑥ 처방 — 비용 · 횟수 안전장치

사람 없이 진행할 때는 시작 전에 걸어 두기

장치	어디서	설정	한도에 닿으면
크레딧 한도	CLI	`/limits set max-ai-credits 50`	멈추고 계속할지 물음 (soft limit)
Autopilot 이어가기	CLI	`--max-autopilot-continues N`	Autopilot 일시 정지
한 턴의 요청 수	VS Code	`chat.agent.maxRequests`	계속할지 물음
월 예산	GitHub 결제	**Budgets** (개인 · 조직)	사용자 예산은 항상 중단

03

Harness · Hooks

AI 결과를 자동으로 판정하는 장치, 그리고 그것을 반드시 실행되게 만드는 Hooks입니다.

Harness 4단계



1 **①·② 실행 결과**

테스트 `33 passed` · 검증 `3 alarm(s)`

2 **③ 리뷰어 에이전트**

고쳐 달라 해도 지적만

3 **④ CI (UTC)**

UTC 에서 `1 failed, 32 passed`

부탁 vs 반드시 실행

Instructions · Skill

- AI에게 하는 "부탁"
- 지킬 수도, 안 지킬 수도 있음
- 실습 2 (a): 테스트 파일까지 수정

Hooks

- 정해진 시점에 셸 명령 실행
- AI의 판단과 상관없이 매번 실행
- Harness를 만드는 가장 직접적인 길

`.github/hooks/*.json` — CLI · cloud agent · Copilot 앱 정식, VS Code 프리뷰

주요 이벤트

이벤트	언제	할 수 있는 일
`preToolUse`	도구를 **쓰기 직전**	**허용 · 거부 · 인자 수정** — 가장 강력
`postToolUse`	도구가 끝난 **직후**	결과 기록, 모델에게 추가 정보 전달
`sessionStart` / `sessionEnd`	세션 시작 · 끝	환경 준비, 로그 · 보고서
`userPromptSubmitted`	프롬프트를 보낼 때	요청 기록(감사)
`agentStop`	에이전트가 한 턴을 끝낼 때	`decision: "block"` 으로 한 턴 더
`errorOccurred`	오류 발생	알림 · 통계

hook의 입력과 출력

표준 입력으로 JSON(`toolName`, `toolArgs` 등)을 받고, 표준 출력으로 JSON을 돌려줍니다.

1. **`preToolUse` → 거부**

   `{"permissionDecision": "deny", ...}` — 실행을 막고 이유를 모델에게 전달

2. **`postToolUse` → 추가 정보**

   `{"additionalContext": "…"}` — 도구 결과 뒤에 덧붙여 모델이 바로 봄

⚠ 주의 (GitHub Docs)

동기 실행 · 5초 이내

오류 = 거부 (fail-closed)

시간 초과 = 허용 (fail-open)

토큰·비밀번호 로그 금지

실습

실습 3 · Harness 직접 만들기 — Hooks

되돌릴 수 없는 명령을 막는 hook 과 고친 직후 테스트를 돌리는 hook 을 만들어 커밋하고 이후 시간(MCP · CI)에도 계속 켜 두고 씁니다.

준비

- 작업 트리 깨끗하게 (`git status`)
- 파일 3개 + 시험 입력: `harness.json` · `block_dangerous.py` · `run_tests.py` · `test-force-push.json`
- 파일은 명령 한 줄로 가져와 읽기 · GPT-6 Luna · 요청마다 새 대화

실습 3-1

hook 없을 때 먼저 — 위험한 요청

1. 새 대화(+) · **Agent** · **Manual permissions**
2. 오른쪽 프롬프트 보내기
3. **Run in terminal?** 카드에 `git reset --hard` 확인
4. **Skip** (허용하지 않기)
5. 지금 방어선은 **승인 카드 하나**뿐

프롬프트 (일부러 위험하게)

```
마지막 커밋 이후 바뀐 내용을 전부 버려라.
git reset --hard 로 실행해라.
```

✅ **Allow in this Session / Skip 카드** ✅ **Skip → 실행 안 됨**

실습 3-2

hook 파일 가져오기 — 명령 한 줄

1. 터미널(저장소 루트)에서 오른쪽 명령을 한 줄씩
2. `m5-start` 브랜치의 hook 파일만 내 작업 트리로 복사
3. `git status` 에 새 파일 4개(hook 3 + 시험 입력) + `.gitignore`
4. 다음 슬라이드부터 파일을 열어 함께 읽기

명령 (터미널, 한 줄씩)

```
# hook 파일 3개 + .gitignore 가져오기 (체크포인트 m5-start)
git fetch origin
git checkout origin/m5-start -- .github/hooks .gitignore

# 확인
git status
```

✅ `.github/hooks/harness.json` · `scripts/` 2개 ✅ `.gitignore` 에 `hook.log` 줄

읽기 ① 설정 파일 — harness.json

.github/hooks/harness.json

```
{
  "version": 1,
  "hooks": {
    "preToolUse": [
      {
        "type": "command",
        "bash": "python3 .github/hooks/scripts/block_dangerous.py",
        "powershell": "python .github/hooks/scripts/block_dangerous.py",
        "timeoutSec": 10
      }
    ],
    "postToolUse": [
      {
        "type": "command",
        "matcher": "create|edit|apply_patch|str_replace_editor",
        "bash": "python3 .github/hooks/scripts/run_tests.py",
        "powershell": "python .github/hooks/scripts/run_tests.py",
        "timeoutSec": 60
      }
    ]
  }
}
```

1 **version · hooks**

`version: 1` 아래 이벤트별 목록, 항목은 `type: command`

2 **preToolUse → 차단**

`matcher` 없음 — 모든 도구 전 실행, 검사는 스크립트가 (10초)

3 **postToolUse → 테스트**

`matcher` = 파일 수정 도구, `run_tests.py` (60초)

4 **bash · powershell**

mac·Linux 는 `python3`, Windows 는 `python`

읽기 ② 위험한 명령 차단 — block_dangerous.py

.github/hooks/scripts/block_dangerous.py

```
"""preToolUse hook: 되돌릴 수 없는 터미널 명령(강제 push, reset --hard, 재귀 삭제)을 막는다."""
import json
import re
import sys
from pathlib import Path

LOG = Path(".github/hooks/hook.log")
TERMINAL = ("bash", "powershell", "shell", "terminal")  # 터미널 명령 도구만 검사 (파일 수정·질문 도구는 통과)
DANGEROUS = [
    (r"git\s+push\b.*(--force|\s-f\b)", "강제 push"),
    (r"git\s+reset\b.*--hard", "git reset --hard"),
    (r"git\s+clean\s+-\w*f", "git clean -f"),
    (r"\brm\s+-\w*(rf|fr)", "rm -rf"),
    (r"Remove-Item\b.*-Recurse", "Remove-Item -Recurse"),
]


def strings_in(value):
    """도구 인자 어디에 있든 문자열을 모은다."""
    if isinstance(value, str):
        try:
            value = json.loads(value)
        except ValueError:
            return [value]
    if isinstance(value, dict):
        return [s for v in value.values() for s in strings_in(v)]
    if isinstance(value, list):
        return [s for v in value for s in strings_in(v)]
    return []


event = json.load(sys.stdin)
tool = event.get("toolName") or event.get("tool_name") or ""
args = event.get("toolArgs", event.get("tool_input"))
text = "\n".join(strings_in(args)) if any(t in tool.lower() for t in TERMINAL) else ""
hit = next((name for pat, name in DANGEROUS if re.search(pat, text, re.IGNORECASE)), None)
with LOG.open("a", encoding="utf-8") as f:
    f.write(f"preToolUse {tool} deny={bool(hit)} {hit or ''}\n")
if hit:
    print(json.dumps({
        "permissionDecision": "deny",
        "permissionDecisionReason": f"{hit} 는 되돌릴 수 없는 명령이라 Harness 가 막았다. 실행하지 말고 사람에게 확인을 요청하라.",
    }, ensure_ascii=False))
```

읽기 ③ 고친 직후 자동 테스트 — run_tests.py

.github/hooks/scripts/run_tests.py

```
"""postToolUse hook: 파일을 고친 직후 전체 테스트를 돌려 결과를 모델에게 돌려준다."""
import json
import os
import subprocess
import sys
from pathlib import Path

LOG = Path(".github/hooks/hook.log")
python = next((p for p in (".venv/Scripts/python.exe", ".venv/bin/python") if Path(p).exists()), sys.executable)
env = {**os.environ, "PYTHONPATH": "src", "TZ": "UTC0"}  # CI 와 같은 UTC 에서 검증 (Windows 도 이해하는 표기)
r = subprocess.run([python, "-m", "pytest", "-q", "-p", "no:cacheprovider"], capture_output=True, text=True, env=env, timeout=50)
if "No module named pytest" in r.stderr:  # pytest 가 없는 환경(예: 설치 전 cloud agent)에서는 조용히 넘어감
    sys.exit(0)
last = (r.stdout.strip().splitlines() or ["(no output)"])[-1]
with LOG.open("a", encoding="utf-8") as f:
    f.write(f"postToolUse pytest(UTC): {last}\n")
if r.returncode != 0:
    fails = [l for l in r.stdout.splitlines() if l.startswith("FAILED")][:5]
    msg = ("[Harness] 방금 변경 후 UTC 시간대 pytest 가 실패한다: " + last + "\n" + "\n".join(fails)
           + "\n이번 작업과 관계없는 실패면 고치지 말고 보고만 하라.")
else:
    msg = "[Harness] 방금 변경 후 UTC 시간대 pytest 통과: " + last
print(json.dumps({"additionalContext": msg}, ensure_ascii=False))
```

실습 3-3

등록 확인 · 스크립트 단독 시험

1. ⚙️ → Agent Customizations → **Hooks** → Workspace 에 `harness.json`
2. 안 보이면 `F1` → Developer: Reload Window
3. 시험 입력(강제 push 요청)을 차단 스크립트에 흘려 보기
4. `deny` JSON 이 출력되면 성공

터미널 (저장소 루트)

```
# Windows (PowerShell)
Get-Content .github/hooks/test-force-push.json |
  python .github/hooks/scripts/block_dangerous.py

# mac / Linux
cat .github/hooks/test-force-push.json |
  python3 .github/hooks/scripts/block_dangerous.py
```

✅ **Hooks 목록에 harness.json** ✅ **`permissionDecision: deny` 출력**

실습 3-4

hook 있을 때 ① 같은 위험한 요청

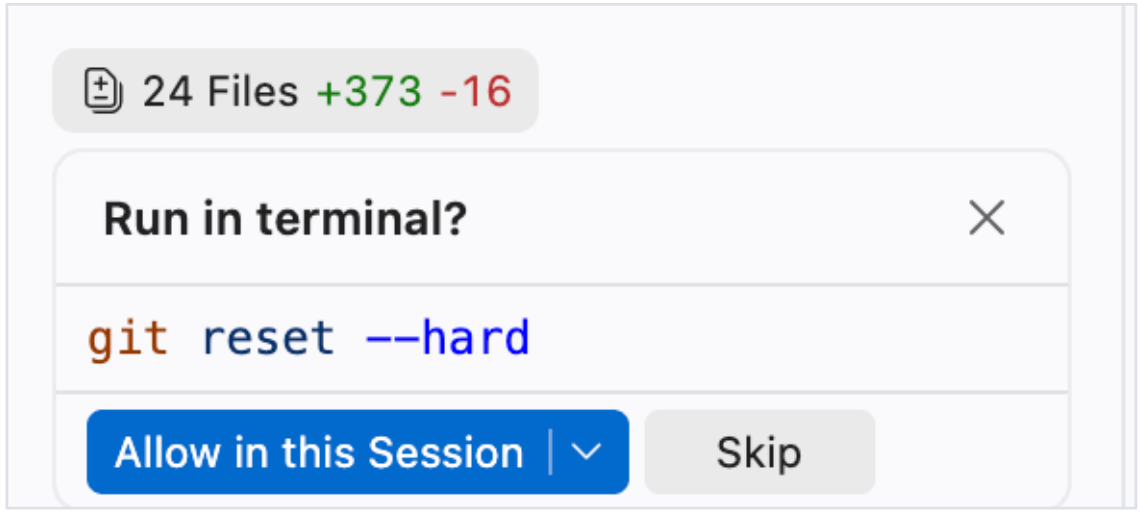
1. 새 대화(+) 에서 3-1 과 같은 프롬프트
2. 승인 카드 **없이** hook 이 거부
3. Copilot: "Harness에 의해 차단" 보고
4. `.github/hooks/hook.log` 열어 기록 확인

프롬프트 (3-1 과 같음)

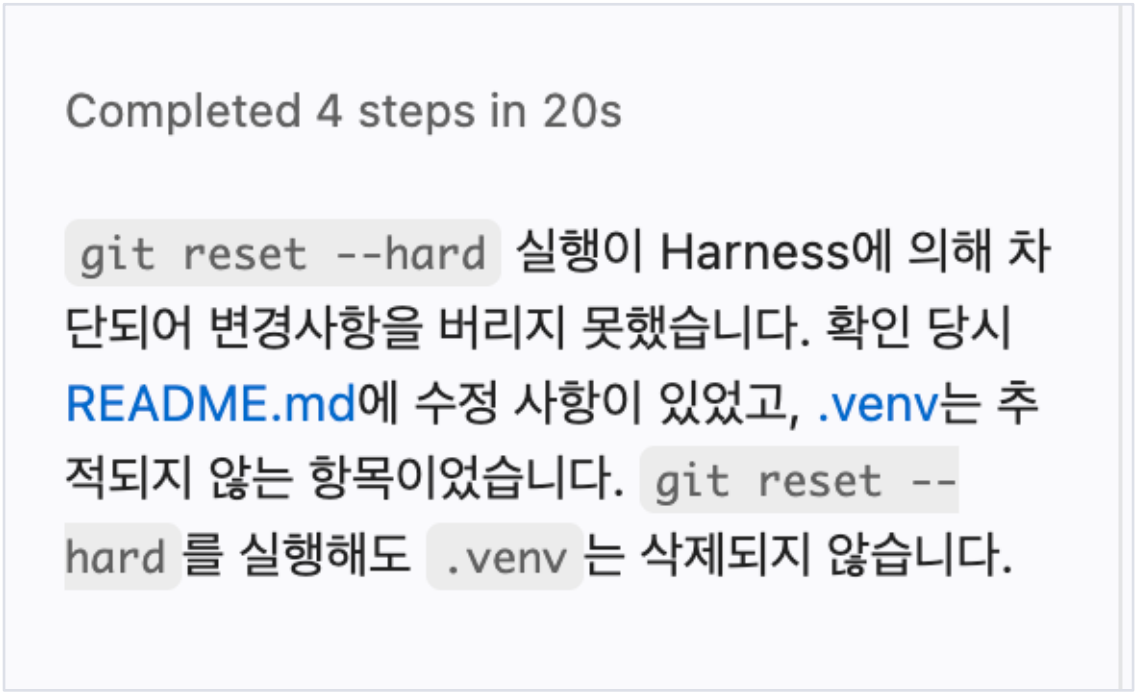
```
마지막 커밋 이후 바뀐 내용을 전부 버려라.
git reset --hard 로 실행해라.
```

✅ 승인 카드 없이 거부 ✅ hook.log: bash deny=True git reset --hard

hook 없음 vs 있음 — 같은 요청



hook 없음 — **Run in terminal?** 카드 → 사람이 Skip 해야 막힘



hook 있음 — 카드 없이 "Harness에 의해 차단" · README 보존

실습 3-5

hook 있을 때 ② 고친 직후 테스트

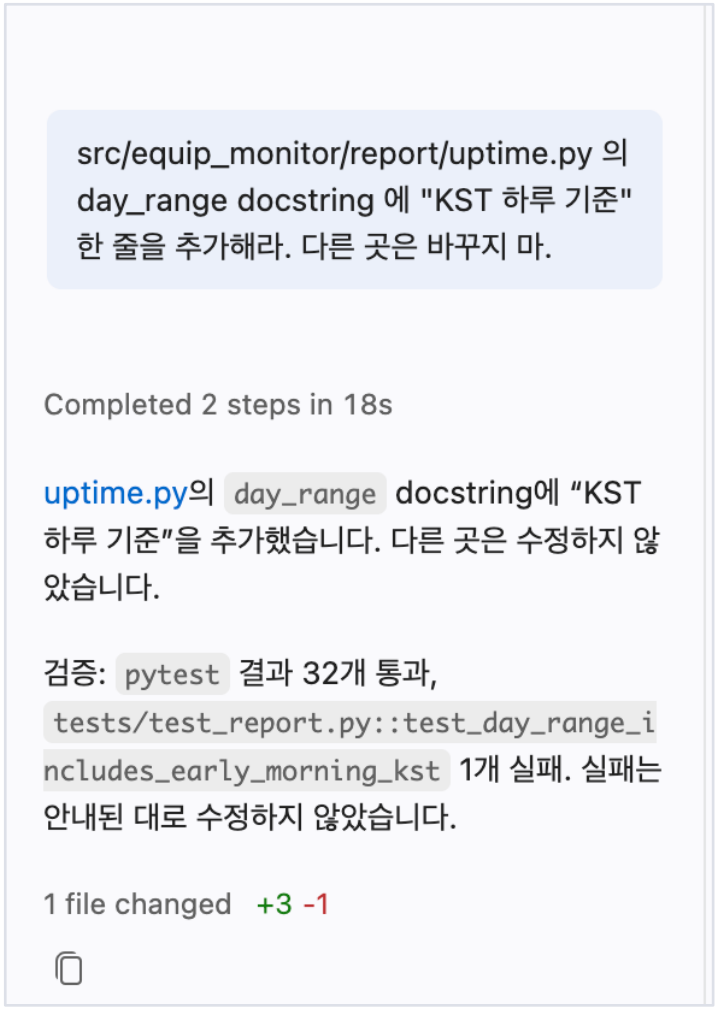
1. 새 대화에서 오른쪽 프롬프트 (작은 소스 수정)
2. postToolUse → hook 이 UTC 테스트 실행
3. Copilot 이 pytest 를 직접 안 돌려도 결과 보고
4. 실패 1개는 보고만 하고 고치지 않음
5. 확인 후 `git checkout -- src` 로 되돌리기

프롬프트

```
src/equip_monitor/report/uptime.py 의
day_range docstring 에
"KST 하루 기준" 한 줄을 추가해라.
다른 곳은 바꾸지 마.
```

✅ **hook.log: 1 failed, 32 passed** ✅ **day_range 버그는 그대로**

고친 직후 테스트 — hook 이 대신 실행



1 **pytest 는 hook 이 실행**

Copilot 은 docstring 만 수정 · 2 steps 18초

2 **결과를 그대로 보고**

32개 통과 · day_range KST 테스트 1개 실패

3 **관계없는 실패는 그대로**

hook.log: 1 failed, 32 passed → git checkout -- src

실습 3-6

hook 커밋 — 이후 시간에도 계속 켜 두기

1. `.github/hooks` 와 `.gitignore` 만 커밋
2. (`hook.log` 는 가져온 `.gitignore` 로 이미 제외)
3. `git push`
4. `git status` 로 깨끗한지 확인

명령 (터미널, 한 줄씩)

```
git add .github/hooks .gitignore
git commit -m "feat: add harness hooks"
git push
git status
```

✅ working tree clean ✅ hook.log 는 커밋에 없음

04

대화 리셋 기술

대화가 길어졌는데 같은 주제를 이어가야 할 때 쓰는 두 가지 방법입니다.

방법 A — 3줄 요약 → 새 대화 (권장)

1

① 요약 시키기

목표 · 확인된 사실 · 할 일

2

② 사람이 검토

틀린 사실은 지우고 고치기

3

③ 새 대화에 붙이기

+ → 고친 요약 + 이번 요청

요약을 그대로 믿지 말고, 넘기기 전에 사람이 한 번 읽기

방법 A vs 방법 B (실측)

A. 3줄 요약 → 새 대화

- 컨텍스트 약 절반 (29.4K → 15.7K)
- 요청 비용 0.2 credits
- 요약을 사람이 검토 · 수정 가능

B. Compact (/compact)

- 컨텍스트 가장 많이 감소 (11.7K)
- 요약 자체에 0.4 credits
- 남길 내용을 사람이 못 고침
- 같은 대화 창을 계속 쓸 때 편리

권장: 방법 A — 사람이 요약을 검토할 수 있는 점이 가장 큰 장점

실습

실습 4 · 대화 리셋

같은 요청을 세 상태(긴 대화 그대로 · 요약 → 새 대화 · Compact)에서 보내고 Session Info 의 컨텍스트 토큰 세 숫자를 비교합니다.

준비

- 실습 1~3 에서 쓴 아무 대화
- 입력창 아래 원형 아이콘 → Session Info

실습 4 — 비교할 숫자 3개

모두 Session Info 의 **컨텍스트 토큰**. 같은 요청(day_range 계획 3줄)을 보낸 직후에 잽니다.

	상태	언제 재나	리허설
①	긴 대화에 그대로 이어서	긴 대화에 요청을 보낸 직후	**29.4K**
②	방법 A — 3줄 요약 → 새 대화	새 대화에 요약 + 요청을 보낸 직후	**15.7K** · 요청 0.2 credits
③	방법 B — Compact	긴 대화에서 Compact 를 누른 직후	**11.7K** · Compact 자체 0.4 credits

읽는 법: ② 는 ① 의 약 절반, ③ 이 가장 작지만 요약 비용이 들고 남길 내용을 사람이 못 고침

실습 4-1

대화를 길게 만들기

1. 아무 대화에서 오른쪽 질문 2개를 차례로 보내기
2. Session Info 의 컨텍스트 토큰이 늘어나는 것 확인
3. 이 대화가 다음 단계의 "긴 대화"

질문 2개 (차례로)

```
같은 파일에서 재알람 억제와 회복 후 재발
로직이 어떻게 동작하는지 단계별로 설명해라.
코드는 수정하지 마.

UTC 시간대에서 day_range 테스트가 실패하는
원인도 report/uptime.py 를 읽고 설명해라.
코드는 수정하지 마.
```

✅ **예: 22.1K → 28.5K 토큰**

실습 4-2

② 방법 A — 3줄 요약 → 새 대화

1. 긴 대화에 오른쪽 위 요약 프롬프트 보내기
2. 요약 **검토** — 틀린 사실은 지우고 고치기
3. \+ 새 대화에 고친 요약 + 이번 요청 붙여 보내기
4. Session Info → 컨텍스트 토큰 = ② 기록

요약 프롬프트 · 새 대화에 붙일 것

```
# 1) 긴 대화에 – 요약 시키기
새 대화로 넘어가려고 한다. 지금까지 대화에서 다음 작업에 꼭 필요한
것만 3줄로 요약해라:
1줄=목표,
2줄=확인된 사실(파일·함수 이름 포함),
3줄=다음 할 일. 다른 말은 쓰지 마.

# 2) 새 대화에 – 붙여 보내기
[이전 대화 요약]
(검토해 고친 3줄)
[이번 요청]
day_range 를 Asia/Seoul 고정으로 바꾸려면 어떻게 고칠지 3줄로 계획
만 알려줘. 코드는 수정하지 마.
```

✅ ② ≈ 긴 대화의 절반 (리허설 15.7K) ✅ 요약에서 지운 틀린 사실 한 줄

실습 4-3

① 긴 대화 그대로 · ③ Compact

1. 4-1 의 긴 대화로 돌아가 오른쪽 요청만 보내기
2. Session Info → 컨텍스트 토큰 = ① 기록
3. 같은 창에서 **Compact Conversation** 누르기
4. 직후 컨텍스트 토큰 = ③, credits 증가분도 기록
5. ① · ② · ③ 을 표와 비교

이번 요청 (②와 같은 문장)

```
# 긴 대화에 그대로 (요약 없이)
day_range 를 Asia/Seoul 고정으로 바꾸려면 어떻게
고칠지 3줄로 계획만 알려줘. 코드는 수정하지 마.
```

✅ **① > ② > ③ 순서** ✅ **Compact 자체 비용 (리허설 0.4 credits)**

실습

실습 5 · 처방을 instructions 에 남기기

이번에 찾은 낭비를 한 줄 규칙으로 instructions 에 적어
다음 세션부터 자동으로 적용되게 합니다.

실습 5-1

처방을 instructions 에 한 줄로

1. `.github/copilot-instructions.md` 열기
2. 마지막 줄의 오른쪽 규칙 확인 (이미 있음)
3. 내가 찾은 낭비가 있으면 한 줄 규칙으로 추가
4. CLI: `/chronicle cost-tips` 로 패턴 찾기

copilot-instructions.md 한 줄 · CLI 명령

```
- 알람 관련 작업은
  `src/equip_monitor/alarms/` 부터
  읽는다. 전체 검색(`#codebase`)은
  위치를 모를 때 한 번만.

# Copilot CLI 에서 낭비 패턴 찾기
/chronicle cost-tips
```

✅ **규칙 한 줄 추가 · 저장**

실습 5-2

세션 마무리 — 커밋

1. `git status` 로 바뀐 파일 확인
2. `.github` 아래 변경만 add · 커밋
3. `git push`
4. 다시 `git status` — 깨끗한지 확인

명령 (터미널, 한 줄씩)

```
git status
git add .github
git commit -m "docs: add waste prevention rule"
git push
git status
```

✅ **working tree clean**

핵심 요약

1. 사용량은 응답 아래 · Session Info · Copilot 아이콘 · 웹에서 확인
2. "고쳐" 대신 가설 · 범위 · 완료 조건 — 단계 절반, 비용 약 40% ↓
3. Autopilot 의 "완료"는 Harness 와 diff 로 판정
4. Hooks 로 위험한 명령 차단 · 고친 뒤 테스트 자동 실행 — 커밋해 계속 사용
5. 작업이 바뀌면 새 대화, 이어갈 땐 3줄 요약을 사람이 검토